

What each stage does

TATVA takes a trained ONNX model and produces a standalone C99 program that runs on a bare-metal RISC-V core, then measures it. This is what happens at each of the five stages, what each one needs, and what each one can tell you.

01 Input model + target chip	02 Analyze graph + operators	03 Map operator → lowering	04 Optimize choose passes	05 Generate build + measure
--	--	--	---	---

Stages 01–04 run entirely inside the app. Stage 05 is the only one that shells out to another program — the RISC-V cross-compiler and the QEMU emulator, both bundled with this build.

Everything runs on the machine in front of you

No account, no upload, no telemetry, no network call. A model never leaves the disk it is on, and a complete five-stage run needs no connection at any point. The one exception is opt-in and off by default: the Assistant panel talks to whichever LLM provider you configure yourself.

Input — load the model, pick the chip

Establishes the two facts everything downstream depends on: what you are compiling, and what you are compiling it for.

NEEDS

An `.onnx` file. Nothing else — no toolchain, no network.

PRODUCES

A loaded model, its size and SHA-256, and a selected RISC-V target.

TYPICAL TIME

Under a second.

What happens

The file is read from disk in the app's own process. TATVA records its size, computes a SHA-256 (the first 12 hex digits are shown, so you can tell two similar exports apart), and counts the operator nodes in the ONNX graph without importing it yet. Four sample models ship with the app, so a first run has something to compile.

ONNX only — TATVA does not convert

A `.pt`, `.pth` or `.keras` file can be selected, and the app will tell you it cannot read it. Export to ONNX from PyTorch or TensorFlow first (`torch.onnx.export`, `tf2onnx`) and load that file. This is a deliberate boundary: a conversion TATVA performed silently would be a second place for the model to change before anything is measured.

Choosing the target

The list is read live from the compiler's target registry, so it is always exactly what the code generator will accept. The target decides the instruction set, the ABI, and which multilib the linker pulls in — it is not a label, it changes the binary.

TARGET	-MARCH	WIDTH	USE IT WHEN
RV32IMC	rv32imc	32-bit	Small microcontroller class core; smallest of the 32-bit options.
RV32IMAC	rv32imac	32-bit	As above, with atomics.
RV64GC	rv64gc	64-bit	Default. General-purpose application core with hardware floating point.
RV64IMAFDC	rv64imafdc	64-bit	The same ground as RV64GC, spelled out extension by extension.
RV64GCV	rv64gcv	64-bit	EXPERIMENTAL Vector extension present. Code generation does not yet emit vector instructions.
RV32EMC	rv32emc	32-bit	EXPERIMENTAL Embedded profile, 16 registers.

What can go wrong here

- **The file is not ONNX.** Export it properly rather than renaming it.
- **The ONNX file is a newer opset than the importer handles.** Stage 02 reports it, with the operator named.

Analyze — read the graph and count what is in it

Turns the model file into a structure TATVA can reason about, and finds the pattern that stage 04 knows how to speed up.

NEEDS

A model from stage 01. No toolchain.

PRODUCES

Operator counts, an operator histogram, and an attention-pattern verdict.

TYPICAL TIME

A few seconds; slower on the first run while the importer loads.

What happens

The ONNX graph is imported into Apache TVM's Relax intermediate representation — the same importer the command line's `tatva analyze` uses, so the app and the CLI cannot disagree. TATVA then counts every operator call, counts how many distinct kinds there are, and builds the histogram shown on the page.

The attention pattern

The one structural thing stage 02 looks for is a **softmax feeding a matrix multiply** — the shape of an attention block. This matters because it is precisely the subgraph the softmax fusion pass in stage 04 rewrites. A model without it is not broken and will compile normally; it simply has nothing for that pass to fuse, and stage 05 will honestly report a 0.00% change.

Reading the histogram

A long tail of one-off operators usually means the export carried shape-manipulation nodes that a cleanup pass would remove. A few operators repeated many times is the normal shape of a transformer block and is what stage 03 will spend its time on.

What can go wrong here

- **An operator the importer does not know.** The failure names it. This is a stage-02 answer to a question that would otherwise have surfaced as a compile error minutes later.
- **A malformed or truncated ONNX file.** Re-export it.

When a run fails, the app also prints a plain-English explanation under the error — what the failure category means and what to do about it — rather than only the exception text.

Map — check every operator against the chosen chip

Answers, before anything is built: can this target actually run this model, and how will each operator be turned into machine code?

NEEDS

Stage 02's analysis, and the target from stage 01.

PRODUCES

A per-operator lowering table and a go / no-go verdict.

TYPICAL TIME

Under a second.

What happens

Every operator found in stage 02 is checked against two things: whether the selected target can execute it at all, and how TATVA's code generator intends to lower it. The result is one row per operator with a verdict.

VERDICT	MEANING
FUSED	Emitted as a single fused kernel rather than as separate operators. This is where the measured gain comes from.
HOT PATH	Supported, and expected to dominate the cycle count. Worth looking at if the result disappoints.
MAPPED	Supported, lowered to a scalar loop nest. Correct, unremarkable.
NO LOWERING	The code generator has nothing to emit for this operator on this target.

Why this stage exists at all

A single **No lowering** row is a stage-05 compile failure waiting to happen. Finding it here costs a second; finding it in stage 05 costs a full code-generation and cross-compile cycle first. If you see one, the usual fixes are to pick a different target, or to re-export the model without the operator in question.

Changing the target here

The mapping is model × target, so going back to stage 01 and choosing a different chip produces a different table. That is the intended way to compare two candidate cores before committing to a build.

Optimize — choose the passes

Selects the graph rewrites applied before code generation. Nothing is built here; this stage only decides what stage 05 will do.

NEEDS

A mapped model. No toolchain.

PRODUCES

A build plan — the exact configuration stage 05 will compile.

TYPICAL TIME

Instant. It is a set of switches.

Softmax fusion AVAILABLE

Replaces the generic softmax — multiple passes over the data, heap-allocated scratch space — with a stack-allocated single-pass kernel that uses Schraudolph's fast exponential approximation. On the attention pattern stage 02 detects, this removes an entire pass over the attention output. **This is the pass the measured improvement comes from.** Its output is checked against host ONNX Runtime in stage 05, so the speed-up is not accepted on trust.

INT8 quantization EXPERIMENTAL

Symmetric QDQ quantization, with activation scales calibrated at the 99.9th percentile. It reduces the binary and the working set — and on a scalar RISC-V core it runs **slower**, not faster.

That is a property of the hardware, not a defect

None of TATVA's targets have an INT8 dot-product instruction. Every quantized multiply is widened back out in software, so you pay conversion cost for arithmetic that was already cheap. Turn this on when SRAM or flash is the binding constraint, and leave it off when latency is. Stage 05 measures it either way rather than asking you to take this on faith.

Describing what you want in plain English EXPERIMENTAL

The text box above the pass cards accepts a sentence — *"this has to fit in SRAM on a 32-bit MCU"*, *"lowest latency, accuracy matters"* — and sets the same switches you can click yourself. It is a deterministic phrase matcher running offline: no model, no network, no API key, and the same sentence always resolves to the same configuration.

- Every change it makes is listed with the phrase that caused it, so you can disagree with it.
- An explicit instruction beats an inferred priority: *"smallest possible, but no quantization"* leaves quantization off.
- Contradictory requests are reported rather than silently resolved: *"smallest and fastest"* says which one it took and why.
- Anything it does not recognise is left at your current setting, and it says so instead of returning defaults.

The build plan

The panel at the bottom of the stage shows the exact configuration stage 05 will compile — target, ABI, passes, and the parity tolerance. It is a summary, not a command to run.

Generate — build the binary and measure it

Emits C, cross-compile it for the target, runs it under emulation, and measures the baseline and the optimized build in the same run.

NEEDS

The RISC-V cross-compiler and QEMU. **Both are bundled** with this build.

PRODUCES

Two ELF binaries, two latency figures, and a numerical parity check.

TYPICAL TIME

Minutes on a first run — two full builds and two emulated runs.

What happens, in order

- **Code generation.** TVM lowers the graph to C99. The build directory gets `operators.c`, `model_run.c`, `main.c` and `model_info.h`, plus `start.S` and the linker script `link.ld`.
- **Cross-compile and link.** `riscv-none-elf-gcc` builds it freestanding — no operating system, no C library startup — producing `model.elf`.
- **Emulated run.** QEMU boots the ELF in system mode. The program reads the cycle counter on the target itself with `rdcycle/rdcycleh`, so the timing is taken inside the machine being measured.
- **The same thing again, without the passes,** to produce the baseline the optimized build is compared against.
- **Parity check.** The output is compared against host ONNX Runtime and reported as a mean squared error, with a pass/fail against the tolerance.

```
// how the binary is built and run
riscv-none-elf-gcc -O2 -march=<target> -mabi=<abi> -mcmodel=medany \
  -ffreestanding -nostdlib -T link.ld start.S *.c -lm -lgcc -o model.elf

qemu-system-riscv64 -M virt -nographic -icount shift=0 -kernel model.elf
```

How to read the latency numbers

They are QEMU system-mode cycle counts, converted to milliseconds at a nominal 100 MHz. They are **emulator cycles, not silicon**. Their job is to compare two builds of the same model against each other under identical conditions — which they do faithfully. They are not a prediction of wall-clock time on a specific chip, and should not be quoted as one.

A real run, for reference

One complete run on the development machine, using the bundled toolchain with the system PATH scrubbed to confirm nothing outside the app folder was involved:

CONFIGURATION	MEAN LATENCY	CHANGE	PARITY MSE	VERDICT
Baseline — FP32, no passes	0.731 ms	reference	—	REFERENCE
Optimized — softmax fusion	0.665 ms	−8.99%	2.94e−5	PASS

Whole run: 5.3 s. Your figures will differ with the model, the target and the machine. A run that reports 0.00% is a real result — it means the passes you chose had nothing to change in that particular graph.

What can go wrong here

- **The toolchain is not found.** Only possible outside the packaged build; Diagnostics shows the resolved path for each binary and offers to install them.
- **An operator had no lowering.** Stage 03 would have said so first.
- **The parity check fails.** The numbers are still shown, flagged as failed. A large MSE with INT8 on is the expected quantization trade-off, not a compiler bug.

Taking the binary to real hardware

What stage 05 hands you, and what has to be true of a board before it will run there.

TATVA compiles and measures under emulation. It does not flash boards, and asking it for a hardware run returns a message saying so along with the path to the ELF — deliberately, rather than pretending an emulated number came from silicon.

What you have after a build

In the stage-05 build directory: `model.elf`, the linked binary, alongside the generated `operators.c`, `model_run.c`, `main.c`, `model_info.h`, `start.S` and `link.ld`. The C sources are the portable artifact — they are ordinary C99 and can be dropped into an existing firmware project or SDK.

What the ELF assumes

ASSUMPTION	DETAIL
Load address	0x80200000
RAM window	128 MB
Entry point	<code>_start</code>
Stack	64 KB, reserved by the linker script
Console output	SBI ecall — <code>a7=0x01</code> console putchar
Shutdown	SBI system reset — <code>a7=0x53525354</code>
Timing	<code>rdcycle</code> / <code>rdcycleh</code>

The load address and SBI are the two things to check first

Because the binary talks to the console through SBI rather than by writing UART registers directly, it is an **S-mode payload for OpenSBI**, not a raw M-mode bare-metal image. It will run unchanged on a board that boots OpenSBI and hands off at 0x80200000 — which covers most Linux-capable RISC-V SoCs and development boards. A microcontroller-class core with no SBI implementation needs two edits: the **MEMORY** origin in `link.ld` changed to that part's RAM base, and the console putchar pointed at its UART.

Getting it onto the board

- **Via a bootloader (most boards).** Copy `model.elf` to SD card or load it over TFTP, and have U-Boot or OpenSBI load it at 0x80200000 and jump to it.
- **Via a debug probe.** `openocd` with your board's config, then `load` and `continue` from GDB. Note the bundled toolchain has GDB removed to save space — use your own.
- **As source, inside your own firmware.** Usually the right answer for a microcontroller: take the generated `.c` and `.h` files, drop them into your existing project, and call the entry point from your own `main`. You keep your linker script, your startup code and your I/O.

What to expect from the numbers

The emulated figure and the silicon figure will not match — QEMU models instruction counts, not caches, memory latency or pipeline behaviour. What does carry across is the *direction and rough scale* of the difference between two builds, because both were measured the same way. Treat stage 05 as the answer to "did this pass help, and roughly how much", and the board as the answer to "how fast is it actually".

Reference

The same pipeline from the command line

The desktop app and the CLI share one implementation. These are the commands that exist:

COMMAND	DOES
<code>tatva doctor</code>	Toolchain health check — the same one the Diagnostics page runs.
<code>tatva setup</code>	Downloads and installs the RISC-V toolchain. Not needed in the packaged build.
<code>tatva targets</code>	Lists the RISC-V targets the code generator accepts.
<code>tatva analyze</code>	Stage 02 — graph and operator analysis.
<code>tatva baseline-test</code>	Builds and measures the unoptimized baseline.
<code>tatva optimize</code>	Stages 04–05 — apply passes, build, measure both.
<code>tatva diagnose</code>	Explains a failure in plain English.
<code>tatva validate</code>	Numerical parity against host ONNX Runtime.
<code>tatva gui</code>	Opens the desktop app.

There is no `tatva compile`. The `$ tatva compile ...` line shown in the stage-05 console is a summary of the configuration being built, not a command to type.

Where the toolchain comes from

TATVA looks for `riscv-none-elf-gcc` and `qemu-system-riscv64` in this order, and Diagnostics shows which copy it resolved:

- Your PATH — if you have your own, it wins.
- The per-user tools directory (`%LOCALAPPDATA%\tatva\toolchains`).
- The `toolchain/` folder shipped beside the app. **This is the one the packaged build uses**, and it is why stage 05 works on a machine with nothing installed.

What this build can and cannot do

STATUS	FEATURES
AVAILABLE	ONNX input · graph and operator analysis · softmax fusion · RV32IMC–RV64GC cross-compilation · QEMU system-mode validation · desktop app in light and dark
EXPERIMENTAL	INT8 quantization · plain-English diagnostics · natural-language configuration · assistant and project scaffolding
ROADMAP	Vector and matrix extensions · wider operator coverage · Renode validation · more boards and cores · hosted service

Experimental means it works but is not yet trustworthy for a production decision. Roadmap means it is not in this build at all.

One rule this project holds to

Every performance figure shown anywhere in TATVA comes from a measurement that actually ran. Nothing is estimated, extrapolated or rounded up, and a stage that produced no measurement leaves its report page empty rather than filling it with a placeholder.